

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ
ЖИВОТНОГО МИРА НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное учреждение
Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

День 12

Тема: «Отладка программы с использованием специализированных средств
отладки»

Цель работы:

Изучить способы отладки программы с использованием языка
программирования Python

Выполнила: Чичерина

Диана Сергеевна,

Студент 24-ИС

2026г.

Вариант 11

день 13/

```
|— main.py          ← ТВОЙ ФАЙЛ (ЭТОТ)
|— src/
|  |— __init__.py   ← ДОЛЖЕН БЫТЬ (пустой)
|  |— domain/
|  |  |— __init__.py ← ДОЛЖЕН БЫТЬ (пустой)
|  |  |— models.py
|  |  |— exceptions.py
|  |— dto/
|  |  |— __init__.py ← ДОЛЖЕН БЫТЬ (пустой)
|  |  |— booking_dto.py
|  |  |— room_dto.py
|  |— repositories/
|  |  |— __init__.py ← ДОЛЖЕН БЫТЬ (пустой)
|  |  |— base.py
|  |  |— hotel_repo.py
|  |  |— room_repo.py
|  |  |— booking_repo.py
|  |— services/
|  |  |— __init__.py ← ДОЛЖЕН БЫТЬ (пустой)
|  |  |— booking_service.py
|  |  |— pricing_service.py
|  |— observers/
|  |  |— __init__.py ← ДОЛЖЕН БЫТЬ (пустой)
|  |  |— price_observer.py
|  |— uow/
|  |  |— __init__.py ← ДОЛЖЕН БЫТЬ (пустой)
|  |  |— unit_of_work.py
|— requirements.txt
```

```
=====
СИСТЕМА УПРАВЛЕНИЯ БРОНИРОВАНИЯМИ - ВАРИАНТ 11
Политика отмены (State Pattern)
=====
```

```
1. СОЗДАНИЕ ОТЕЛЯ
```

```
✓ Отель создан: ID=1, Grand Hotel
```

```
2. СОЗДАНИЕ НОМЕРА
```

```
✓ Номер создан: ID=1, №101
```

```
3. ТЕСТ: БЕСПЛАТНАЯ ОТМЕНА (10 дней до заезда)
```

```
Создано бронирование: ID=1, стоимость=270.0
```

```
Отмена: штраф=0.0, возврат=270.0
```

```
Политика: Бесплатная отмена, дней до заезда: 10
```

```
4. ТЕСТ: ЧАСТИЧНАЯ ОТМЕНА (5 дней до заезда)
```

```
Создано бронирование: ID=2, стоимость=120.0
```

```
Отмена: штраф=60.0, возврат=60.0
```

```
Политика: Частичная компенсация (50%), дней до заезда: 5
```

```
5. ТЕСТ: ПОЛНАЯ ОПЛАТА (2 дня до заезда)
```

```
Создано бронирование: ID=3, стоимость=120.0
```

```
Отмена: штраф=120.0, возврат=0.0
```

```
Политика: Полная оплата (100%), дней до заезда: 2
```

```
=====
✓ ВСЕ ТЕСТЫ ПРОЙДЕНЫ УСПЕШНО!
=====
```

Все работает.

Контрольные вопросы:

1. Какая основная цель выделения сервисного слоя?

Основная цель выделения сервисного слоя — инкапсуляция бизнес-логики в отдельном слое, чтобы контроллеры занимались только обработкой HTTP-запросов, а вся логика приложения была изолирована, тестируема и переиспользуема независимо от интерфейса.

2. Чем отличается DTO от Domain Model?

DTO (Data Transfer Object) — это объект для передачи данных между слоями, который содержит только поля и валидацию, но не содержит бизнес-логики, в то время как Domain Model — это бизнес-сущность, которая инкапсулирует поведение и правила предметной области.

3. Что такое Unit of Work и зачем он нужен?

Unit of Work — это паттерн, который управляет транзакциями и отслеживает изменения объектов в рамках одной бизнес-операции, обеспечивая атомарность: либо все изменения сохраняются через commit, либо все откатываются через rollback при ошибке.

4. В чем разница между паттернами Repository и Service?

Repository отвечает только за доступ к данным и CRUD-операции, а Service содержит бизнес-логику, использует репозитории для получения данных, применяет правила и управляет транзакциями через Unit of Work.

5. Какую проблему решает внедрение зависимостей (DI)?

Внедрение зависимостей решает проблему жёсткой связанности компонентов, позволяя передавать зависимости через конструктор, что упрощает тестирование с использованием мок-объектов и даёт возможность легко заменять реализации, например, переключаться с in-memory репозитория на реальную базу данных.

6. Почему нельзя использовать бизнес-логику в контроллерах?

Бизнес-логику нельзя размещать в контроллерах, потому что это приводит к дублированию кода, усложняет тестирование, делает логику привязанной к HTTP-протоколу и мешает переиспользовать её в других интерфейсах, например, в CLI-командах или очередях сообщений.

7. Какие типы исключений должны быть в сервисном слое?

В сервисном слое должны быть свои кастомные исключения, наследуемые от базового DomainError, например, RoomNotFoundError, BookingConflictError, InvalidDatesError, которые отражают бизнес-ошибки и позволяют обрабатывать их на уровне выше, не смешивая с техническими исключениями.

8. Как протестировать сервис без подключения к реальной БД?

Сервис тестируется с использованием мок-объектов (Mock), которые заменяют репозитории: мы создаём фиктивные данные, подменяем реальные репозитории на моки, проверяем, что сервис вызывает нужные методы с правильными аргументами и возвращает ожидаемые результаты, при этом реальная база данных не используется.

Вывод: На занятии разработана система управления бронированиями отелей с политикой отмены через паттерн State. Создана структура проекта со слоями: domain (модели Booking, Room, Hotel и исключения), repositories (In-Memory доступ), services (бизнес-логика), dto (Pydantic-валидация) и uow (транзакции). Реализован BookingService с методами create, cancel, confirm. Главная особенность — State Pattern для отмены: за 7+ дней штраф 0%, за 3-6 дней — 50%, за 1-2 дня — 100%, в день заезда отмена запрещена. Написаны unit-тесты с моками. Получена тестируемая, расширяемая система, соответствующая SOLID.

